

Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Курсовой проект по курсу

«Операционные системы»

Группа: М8О-215Б-23

Студент: Аралов И.А.

Преподаватель: Миронов Е.С.

Оценка: _____

Дата: 26.12.24

Москва, 2024

Постановка задачи

Вариант 17.

Необходимо сравнить два алгоритма аллокации: алгоритм Списки свободных блоков (bestfit) и алгоритм двойников по следующим характеристикам:

- Фактор использования
- Скорость выделения блоков
- Скорость освобождения блоков
- Простота использования аллокатора

Каждый аллокатор памяти должен иметь функции аналогичные стандартным функциям free и malloc (realloc, опционально). Перед работой каждый аллокатор инициализируется свободными страницами памяти, выделенными стандартными средствами ядра. Необходимо самостоятельно разработать стратегию тестирования для определения ключевых характеристик аллокаторов памяти. При тестировании нужно свести к минимуму потери точности из-за накладных расходов при измерении ключевых характеристик, описанных выше.

Общий метод и алгоритм решения

Использованные системные вызовы:

- mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset) – выделяет непрерывный регион памяти (обычно анонимная маппинг, без файла), возвращает указатель на него.
- munmap(void *addr, size_t length) – освобождает регион памяти, ранее отображённый через mmap.

Цель данной лабораторной работы сравнить две разные стратегии выделения памяти на примере написания двух аллокаторов: аллокатора, использующего список свободных блоков, и аллокатора, использующего метод двойников. Рассмотрим подробнее каждый из алгоритмов, а потом сравним их на различных тестах.

Аллокатор BF основан на поддержке одного или нескольких списков свободных блоков, а также механизме «best fit». При запуске программы ему передаётся крупная непрерывная область памяти, которую он инициализирует единственным большим свободным блоком. Этот блок описывается управляющим заголовком, в котором хранится информация о размере блока, признаке занятости (свободен ли он в данный момент) и указателе на следующий блок в связном списке.

Когда аллокатору BF поступает запрос на выделение памяти, он сканирует свой список свободных блоков и подбирает блок, который наилучшим образом соответствует запрошенному размеру. При этом выбирается блок, у которого «запас» (разница между размером блока и запрошенным размером) оказывается минимально достаточным, то есть это и есть принцип «best fit». Если выбранный блок заметно больше, чем необходимо, то для уменьшения фрагментации аллокатор разделяет его на два: один блок точно по запрошенному размеру, а оставшаяся часть оформляется как новый свободный блок и добавляется в общий список. Указатель, возвращаемый пользователю, сдвинут за границу управляющего заголовка, чтобы пользователь работал только с полезными байтами.

Освобождение памяти в BF сводится к тому, что по переданному указателю вычисляется адрес заголовка и он помечается как свободный. Далее аллокатор старается объединить его с соседними свободными блоками (смотрит, не лежит ли рядом с ним другой свободный блок). Это объединение, называемое coalescing, позволяет избежать накопления множества мелких фрагментов и даёт более крупные свободные области. Сильной стороной такого подхода оказывается относительно высокий «фактор использования» памяти, поскольку при правильной реализации «best fit» и механизма слияния фрагментация уменьшается. С другой стороны, поиск подходящего блока в линейном списке на каждом вызове может становиться довольно затратным, если число блоков велико.

Принцип работы метода двойников: все выделяемые блоки имеют размер в степень двойки, 4, 8, 16, 32, ... байт; размер изначального единого свободного блока (при старте программы) тоже равен степени 2; когда требуется выделить блок размера S : о находим минимальную степень 2 больше S ; о проверяем наличие свободных страниц такого размера, отдаём если есть; о если нет, находим минимальный блок большего размера и рекурсивно расщепляем его до нужного размера. Получившиеся “обрезки” пополняют списки свободных блоков. при освобождении блока: о находим размер блока; о проверяем, свободен ли второй блок (buddy) и если свободен, объединяем их и рекурсивно проверяем вышестоящий (уже объединённый) блок. Размер соседнего блока (двойника, buddy) всегда равен размеру освобождаемого. Причем сосед только один (слева или справа, согласно алгоритму выделения), его адрес легко установить из значения указателя текущего блока и его размера..

Код программы

main.cpp

```
#include <cstdint> // size_t
#include <iostream>

// или как несколько списков (segregated lists) – здесь для наглядности взят
// один список + поиск лучшего блока (best fit). Это не строгое каноническое

// Структура описывает управляющий блок и память
class Allocator_BF
{
    struct Block
    {
        size_t size; // размер данного блока (без учёта заголовка)
```

```

        bool free;        // свободен ли блок

        Block* next;      // односвязный список
    };

private:

    Block *head; // Голова списка

public:

    // Конструктор: realMemory – указатель на область памяти (mmap),
    // memory_size – её полный размер

    Allocator_BF(void* realMemory, size_t memory_size)
    {
        // Инициализируем один большой блок

        head = reinterpret_cast<Block*>(realMemory);

        head->size = memory_size - sizeof(Block);

        head->free = true;

        head->next = nullptr;
    }

    // Выделение памяти

    void* alloc(size_t block_size)
    {
        // Ищем блок с минимальным «избытком» (best fit)

        Block *bestBlock = nullptr;

        Block *cur        = head;

        size_t minDiff = (size_t)-1; // очень большое число

        // 1) Найдём блок best fit

```

```

while (cur != nullptr)
{
    if (cur->free && cur->size >= block_size)
    {
        size_t diff = cur->size - block_size;

        if (diff < minDiff)
        {
            minDiff = diff;

            bestBlock = cur;
        }
    }

    cur = cur->next;
}

// Если подходящего блока не нашлось
if (!bestBlock)
{
    return nullptr;
}

// 2) Попробуем разделить (если достаточно места для заголовка + данных)
if (bestBlock->size >= block_size + sizeof(Block) + 8)
{
    // Создаём новый блок «хвост» из избытка

    char* basePtr = reinterpret_cast<char*>(bestBlock);

    Block *newBlock = reinterpret_cast<Block*>( basePtr + sizeof(Block) +
block_size );

    newBlock->size = bestBlock->size - block_size - sizeof(Block);

    newBlock->free = true;
}

```

```

        newBlock->next = bestBlock->next;

        // Изменяем размер у блока, который отдаём
        bestBlock->size = block_size;

        bestBlock->free = false;

        bestBlock->next = newBlock;
    }

    else
    {
        // Не делим – просто делаем блок занятым

        bestBlock->free = false;
    }

    // 3) Возвращаем указатель на «полезную» область (после заголовка)
    return reinterpret_cast<void*>(bestBlock + 1);
}

// Освобождение памяти
void free(void* ptr)
{
    if (!ptr) return;

    // Получаем заголовок
    Block *block = reinterpret_cast<Block*>(ptr) - 1;

    block->free = true;

    // Попробуем объединять блоки (coalescing) с соседями

    // Для этого пройдемся с начала списка
    Block *cur = head;

    Block *prev = nullptr;

```

```

while (cur)
{
    if (cur == block)
    {
        // Попробуем слить с предыдущим
        if (prev && prev->free)
        {
            // Объединяем prev и cur
            prev->size += sizeof(Block) + cur->size;
            prev->next = cur->next;
            cur = prev; // текущий блок расширен
        }

        // Попробуем слить с следующим
        if (cur->next && cur->next->free)
        {
            Block *tmp = cur->next;
            cur->size += sizeof(Block) + tmp->size;
            cur->next = tmp->next;
        }

        break; // закончили объединение
    }

    prev = cur;
    cur = cur->next;
}

// Подсчитать сколько всего свободно байт
size_t get_free_memory()
{

```

```

        size_t totalFree = 0;

        Block *cur = head;

        while (cur)

        {

            if (cur->free)

            {

                totalFree += cur->size;

            }

            cur = cur->next;

        }

        return totalFree;

    }

};

#include <iostream>

#include <string>

#define MINLEVEL 4

class Allocator_Buddy

{

    union Block

    {

        int size;

        Block *next;

    };

private:

    void *buffer; // буфер

    Block **queues;

```



```

int levels;

bool is_bro(Block* bro1, Block*bro2, int teck_level)
{
    size_t tbro1 = size_t(bro1) - size_t(buffer);
    size_t tbro2 = size_t(bro2) - size_t(buffer);

    size_t mask = 1 << (levels - teck_level);
    mask = ~mask;

    return (tbro1 & mask) == (tbro2 & mask);
}

int log2_top(size_t x)
{
    int log2 = 0;
    size_t x2 = x;
    while (x >>= 1)
    {
        log2++;
    }

    if (x2 != static_cast<size_t>(1) << log2)
    {
        log2++;
    }

    return log2;
}

int log2_down(size_t x)
{

```

```

    int log2 = 0;

    while (x >= 1)

    {

        log2++;

    }

    return log2;

}

```

public:

```

Allocator_Buddy(void *realMemory, size_t memory_size)

{

    levels = log2_down(memory_size);

    int q_mem = sizeof(Block) * (levels - MINLEVEL + 1);

    levels = log2_down(memory_size - q_mem);

    queues = (Block **)realMemory;

    buffer = (char *)realMemory + q_mem;

    queues[0] = (Block *)buffer;

    queues[0]->next = nullptr;

    for (int i = 1; i <= levels - MINLEVEL; ++i)

        queues[i] = nullptr;

}

void *alloc(size_t block_size)

{

    if (levels < log2_top(block_size + sizeof(Block))) // если попросили
больше чем размер аллокатора

        return nullptr;

    size_t required_level = levels - log2_top(block_size + sizeof(Block)); //
определяем какой уровень нам нужен

```

```

        if (required_level > static_cast<size_t>(levels - MINLEVEL)) //
округляем маленький запрос до размера минимального блока в аллокаторе

        required_level = levels - MINLEVEL;

    if (queues[required_level] != nullptr) // в нужной очереди есть подходящий
блок
    {

        Block *ans = (Block *)queues[required_level];

        queues[required_level] = queues[required_level]->next;

        ans->size = required_level;

        return ans + 1;

    }

    int i = required_level - 1;

    // не повезло, находим минимальный непустой уровень

    for (int j = i; j >= -1; j--)

    {

        if (queues[j] != nullptr){ // опааа, нашли очередь с блоком
свободным

            i = j;

            break;

        }

        if (j == -1) // нет свободных блоков

            return nullptr; // памяти нет :(

    }

    while (queues[required_level] == nullptr) // пока в нужной очереди не
появились блоки делим более большие

    {

        Block *big = queues[i]; // убираем из очереди голову

        queues[i] = queues[i]->next;

```

```

    Block *bro1, *bro2;

    bro1 = big;

    bro2 = (Block *)((char *)big + (1 << (levels - i)) / 2);

    bro1->next = bro2;

    bro2->next = queues[i + 1];

    queues[i + 1] = bro1;

    ++i;
}

```

// вышли из цикла, значит доделили что в нашей очереди есть блоки.
Возвращаем его

```

    Block *ans = (Block *)queues[required_level];

    queues[required_level] = queues[required_level]->next;

    ans->size = required_level;

    return ans + 1;
}

```

```

void free(void *block)
{
    if (block == nullptr)

        return;

    Block *cur = (Block *)block - 1; // поняли в каком мы блоке сейчас
находимся

    int teck_level = cur->size;      // нашли текущий уровень

    cur->next = queues[teck_level];
}

```

```
    queues[teck_level] = cur; // добавили в очередь свободных блоков нужного
размера

    // теперь проходимся вверх и пытаемся соединить cur с двойником

    bool bro_find = true;

    while (bro_find && teck_level >= 0)
    {
        bro_find = false;

        cur = queues[teck_level];

        Block *prev = queues[teck_level];

        while (prev != nullptr)
        {
            if (is_bro(cur, prev->next, teck_level))
            {
                bro_find = true;

                Block *uni = std::min(prev->next, cur);

                prev->next = prev->next->next;

                queues[teck_level] = cur->next;

                uni->next = queues[teck_level - 1];

                queues[teck_level - 1] = uni;

                break;
            }

            prev = prev->next;
        }

        --teck_level;
    }
}
```

```

size_t get_free_memory()
{
    long ans = 0;

    for (int i = 0; i < levels - MINLEVEL; ++i)
    {
        Block *cur = queues[i];

        while (cur != nullptr)
        {
            ans += (1 << (levels - i));

            cur = cur->next;
        }
    }

    return ans;
}

};

#include <iostream>

#include <chrono>

#include <vector>

#include <cstdlib>

#include <random>

#include "BF.cpp"

#include "Buddy.cpp"

const size_t MEMORY_SIZE = 4 * 1024 * 1024; // 4 MB

const int NUM_OPERATIONS = 10000;           // Количество операций

const int MAX_BLOCK_SIZE = 128;             // Максимальный размер блока

```

```

// Общая функция для тестирования любого аллокатора

template<typename Allocator>

void test_allocator(const std::string& name, void* memory) {

    Allocator allocator(memory, MEMORY_SIZE);

    std::vector<void*> pointers;

    std::mt19937 rng(42); // Фиксированный seed для воспроизводимости

    std::uniform_int_distribution<size_t> size_dist(1, MAX_BLOCK_SIZE);

    // Тест скорости выделения памяти

    auto start = std::chrono::high_resolution_clock::now();

    for (int i = 0; i < NUM_OPERATIONS; ++i) {

        void* ptr = allocator.alloc(size_dist(rng));

        pointers.push_back(ptr);

    }

    auto duration = std::chrono::high_resolution_clock::now() - start;

    std::cout << "[" << name << "]" Allocation: "

        << std::chrono::duration<double>(duration).count() << " sec\n";

    // Тест скорости освобождения памяти

    start = std::chrono::high_resolution_clock::now();

    for (void* ptr : pointers) {

        allocator.free(ptr);

    }

    duration = std::chrono::high_resolution_clock::now() - start;

    std::cout << "[" << name << "]" Free: "

        << std::chrono::duration<double>(duration).count() << " sec\n";

    // Тест эффективности использования памяти

```

```

size_t total_allocated = 0;

pointers.clear();

for (int i = 0; i < NUM_OPERATIONS; ++i) {

    size_t size = size_dist(rng);

    if (void* ptr = allocator.alloc(size)) {

        total_allocated += size;

        pointers.push_back(ptr);

    }

}

const size_t free_mem = allocator.get_free_memory();

const double utilization = total_allocated / static_cast<double>(MEMORY_SIZE -
free_mem);

std::cout << "[" << name << "]" Utilization: "

    << std::fixed << utilization * 100 << "%\n\n";

// Очистка памяти

for (void* ptr : pointers) {

    allocator.free(ptr);

}

}

int main() {

    // Выделяем память для аллокаторов

    char* bf_memory = new char[MEMORY_SIZE];

    char* buddy_memory = new char[MEMORY_SIZE];

    // Запуск тестов

```



```
test_allocator<Allocator_BF>("Best-Fit", bf_memory);

test_allocator<Allocator_Buddy>("Buddy", buddy_memory);


// Освобождение памяти

delete[] bf_memory;

delete[] buddy_memory;


return 0;

}
```

Протокол работы программы

Тестирование:

[Best-Fit] Allocation: 0.31667 sec

[Best-Fit] Free: 7.2426e-05 sec

[Best-Fit] Utilization: 72.709505%

[Buddy] Allocation: 0.000644 sec

[Buddy] Free: 0.000210 sec

[Buddy] Utilization: 20.625335%